

ورود به دنیای هوش مصنوعی

دوره‌ی مقدماتی و رایگان برای عموم علاقه‌مندان

جلسه‌ی دهم: مدل‌های زبانی بزرگ – پایان سفر، آغاز راه

علی شریفی زارچی

رئیس کمیته‌ی علمی بین‌المللی المپیاد جهانی هوش مصنوعی

عضو کمیته‌ی علمی بین‌المللی المپیاد جهانی کامپیوتر

مرور جلسه‌ی قبل

- معنا به مثابه مختصات – شاه – مرد + زن $\approx$ ملکه: Embedding

- توجه (Attention): هر کلمه به کلمات مرتبط نگاه می‌کند

- ترنسفورمر: پشته‌ای از لایه‌های توجه – موازی و قدرتمند

- زیرکلمه‌ها (توکن‌ها): واحد کار مدل‌های زبانی

- امروز: همه‌ی ده جلسه، سوار بر هم – LLM

آناتومی کامل یک بلوک ترنسفورمر

هر بلوک دو زیرلایه دارد و دور هر زیرلایه یک میان‌بر بسته شده است:



شبکه‌ی پیش‌خور معمولا بُعد را ۴ برابر می‌کند و برمی‌گرداند: $512 \leftarrow 2048 \leftarrow 512$

ماسک علی روی جمله‌ی کامل

همان ماتریس، این‌بار برای یک مدل مولد: هر توکن فقط تا خودش اجازه‌ی نگاه دارد.

پرسشگر ↓	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزانده
بازیکن	۱۰۰									
توپ	۱۸	۸۲								
را	۶	۸۳	۱۱							
به	۲۰	۲۰	۲۰	۴۰						
دروازه	۷	۷	۷	۲۲	۵۷					
زد	۳۲	۳۱	۳	۳	۱۹	۱۲				
و	۱۰	۱۰	۱۰	۱۰	۱۰	۳۱	۱۹			
آن	۱۲	۲۰	۱	۱	۵۷	۳	۱	۵		
را	۴	۴	۴	۴	۴	۴	۴	۶۴	۸	
لرزانده	۳	۳	۳	۳	۲۵	۳	۳	۳۹	۹	۹

خانه‌های آینده حذف و بقیه دوباره نرمال می‌شوند؛ سطر اول تنها یک گزینه دارد: ۱۰۰٪ خودش.

و دقت کنید: در این حالت، توکن «آن» دیگر «لرزانده» را نمی‌بیند — باید فقط از گذشته تصمیم بگیرد.

معماری کامل: از متن خام تا پیش‌بینی

قطعه‌ها را روی هم بچینیم – این کل یک مدل زبانی است:



همین و بس – تفاوت مدل‌های غول‌پیکر با این نمودار، فقط در عدد N و ابعاد است.

سه شکل استفاده از ترنسفورمر

رمزگذار-رمزگشا

T5 و مدل‌های ترجمه
با Cross-Attention
کار: تبدیل دنباله به دنباله

فقط رمزگشا

GPT و مدل‌های مولد
با ماسک علّی
کار: تولید متن

فقط رمزگذار

BERT و خانواده‌اش
بدون ماسک، دید دوطرفه
کار: فهم و دسته‌بندی

هر سه از همان بلوک ساخته شده‌اند، تفاوت فقط در ماسک و نحوه‌ی اتصال است.

سوال کلیدی

مدلی مثل ChatGPT از کجا می‌آید؟
چه کسی و چگونه آن را می‌سازد؟

خط تولید یک دستیار هوشمند

سه مرحله‌ی کاملاً متفاوت، با داده و هدف متفاوت:



مرحله‌ی اول «دانش» می‌سازد، مرحله‌ی دوم «رفتار»، مرحله‌ی سوم «سلیقه».

مرحله‌ی صفر: داده از کجا می‌آید؟

منبع	سهم تقریبی	چرا مهم است
خزش وب (Common Crawl)	۶۰٪	تنوع عظیم — اما پر از زباله
کتاب‌ها و مقالات	۱۵٪	متن بلند و منسجم، زبان دقیق
ویکی‌پدیا	۵٪	دانش ساختاریافته و باکیفیت
کد (گیت‌هاب)	۱۵٪	منطق، ساختار — و بهبود استدلال!
گفت‌وگوها و انجمن‌ها	۵٪	زبان محاوره و پرسش و پاسخ

نکته‌ی جالب: افزودن کد به داده‌ی آموزشی، توانایی استدلال مدل در مسائل غیرکدی را هم بالا می‌برد.

مقیاس: مدل‌های امروزی روی چند تریلیون توکن آموزش می‌بینند — معادل میلیون‌ها کتاب.

پاک‌سازی داده: کار کثیف پشت صحنه

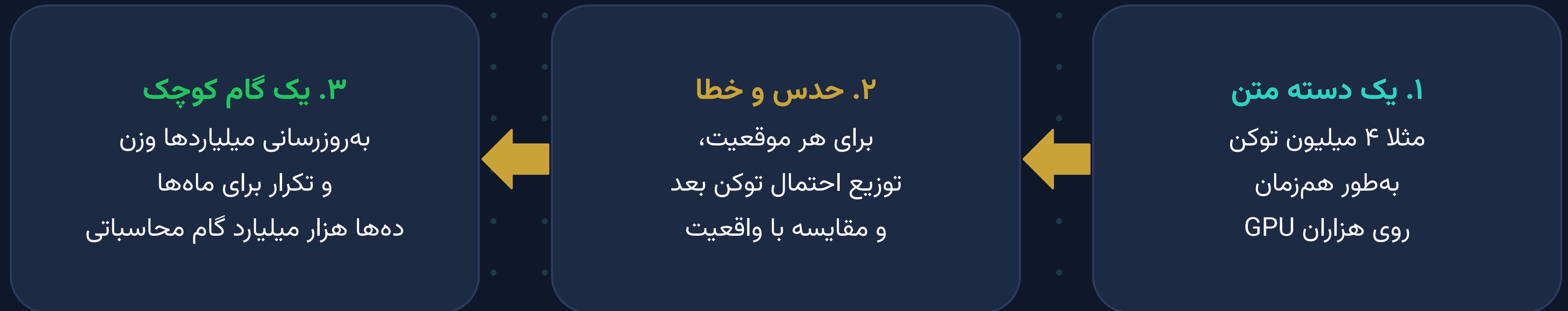
کیفیت داده مهم‌تر از حجم آن است – این مراحل روی پتابایت‌ها متن اجرا می‌شود:



یک قاعده‌ی تجربی: بهبود کیفیت داده معمولاً بیشتر از بزرگ‌کردن مدل جواب می‌دهد.

پیش‌آموزش: میلیاردها بار حدس توکن بعدی

همان تابع هزینه‌ی جلسه‌ی هفتم، فقط در مقیاسی نجومی:



هیچ انسانی این داده را برچسب نزده است – متن، خودش هم سوال است هم جواب.

هزینه‌ی یک دور آموزش مدل بزرگ: ده‌ها تا صدها میلیون دلار.

قانون مقیاس: چقدر داده برای چقدر مدل؟

نسبت	توکن بهینه	اندازه‌ی مدل
۲۰ برابر	۱۴۰ میلیارد	۷ میلیارد پارامتر
۲۰ برابر	۱/۴ تریلیون	۷۰ میلیارد پارامتر
۲۰ برابر	۸ تریلیون	۴۰۰ میلیارد پارامتر

درس Chinchilla: خیلی از مدل‌های اولیه بزرگ ولی «کم‌غذا» بودند؛ داده‌ی بیشتر بهتر از پارامتر بیشتر.

امروز حتی فراتر می‌روند: مدل‌های کوچک‌تر با داده‌ی بسیار بیشتر، برای ارزان‌شدن استفاده.

زیرساخت: چرا این کار سخت است

پایداری

در ماه‌ها آموزش، سخت‌افزار خراب می‌شود
ذخیره‌ی مرتب وضعیت (checkpoint)
و ادامه از آخرین نقطه

موازی‌سازی

داده روی کارت‌های مختلف
لایه‌ها بین کارت‌ها تقسیم
هماهنگی دائمی بین هزاران کارت

حافظه

مدل ۷۰ میلیاردی با دقت کامل:
۲۸۰ گیگابایت فقط برای وزن‌ها
در یک GPU جا نمی‌شود

به همین دلیل فقط چند شرکت در دنیا مدل پایه‌ی بزرگ می‌سازند — اما همه می‌توانند از آن‌ها استفاده کنند.

خروجی مرحله‌ی اول: مدل پایه (Base Model)

- مدل پایه فقط «ادامه‌دهنده‌ی متن» است، نه دستیار
- اگر بنویسید «پایتخت فرانسه کجاست؟» ممکن است ادامه بدهد: «پایتخت آلمان کجاست؟»
- چون در اینترنت، سوال‌ها معمولا کنار سوال‌های دیگر می‌آیند!
- دانش عظیمی دارد، اما نمی‌داند از او چه انتظاری می‌رود
- برای دستیار شدن، به دو مرحله‌ی بعدی نیاز دارد

مرحله‌ی دوم: آموزش دستیاری (SFT)

به مدل ده‌ها هزار گفت‌وگوی نمونه نشان می‌دهیم — با قالبی مشخص:

```
<|system|>
تو یک دستیار مفید و دقیق فارسی‌زبان هستی.

<|user|>
پایتخت فرانسه کجاست؟

<|assistant|>
پایتخت فرانسه، شهر پاریس است.
<|end|>

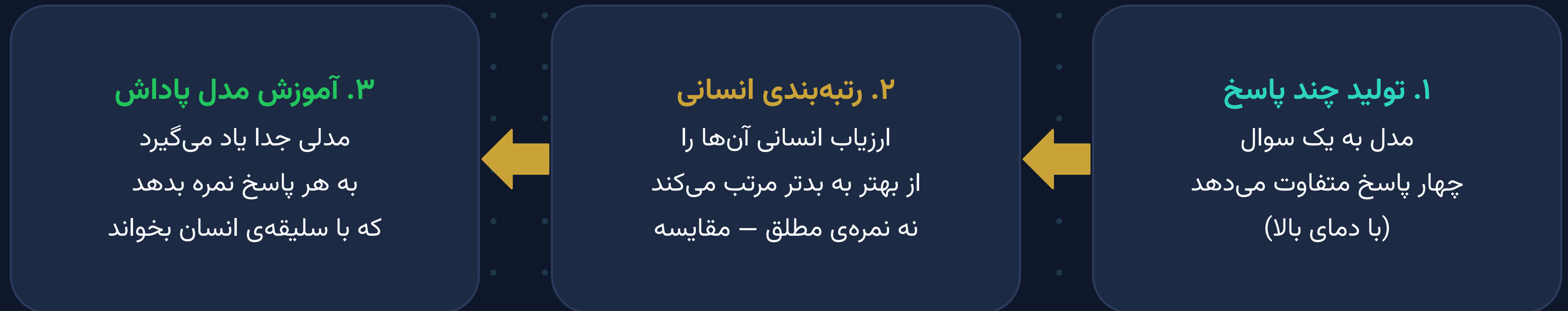
# مدل فقط روی بخش پاسخ دستیار
# آموزش می‌بیند (loss فقط روی آن قسمت)
```

این نشانه‌های ویژه، توکن‌های واقعی مدل‌اند
مدل یاد می‌گیرد بعد از assistant، پاسخ بدهد نه سوال بعدی
کیفیت این چند ده هزار نمونه، شخصیت مدل را می‌سازد
نمونه‌ها را انسان‌های آموزش‌دیده می‌نویسند — گران و زمان‌بر
همین مرحله، «مدل پایه» را به «دستیار» تبدیل می‌کند

پرامپت سیستمی که در API می‌نویسید، دقیقاً در همین جایگاه می‌نشیند.

مرحله‌ی سوم: بازخورد انسانی – ساخت مدل پاداش

چطور «کیفیت پاسخ» را به عدد تبدیل کنیم؟



نکته‌ی ظریف: مقایسه‌ی دو پاسخ برای انسان بسیار راحت‌تر از نمره‌دادن مطلق است.

و همین‌جاست که سلیقه‌ی ارزیاب‌ها وارد مدل می‌شود – ریشه‌ی «چاپلوسی» که در جلسه‌ی اول دیدیم.

سوال کلیدی

کل کاری که ChatGPT می‌کند:
«حدس توکن بعدی» است.

مدل زبانی چیست؟

- مدلی که احتمال «توکن بعدی» را حدس می‌زند – همین!
- «هوا امروز خیلی ...» ← سرد ۳۵٪، گرم ۳۰٪، خوب ۲۰٪، بنفش ۱/۰٪
- با کیبورد گوشی‌تان سال‌هاست یک مدل زبانی کوچک دارید
- متن بلند؟ توکن حدس بزن، اضافه کن، دوباره حدس بزن – و همین‌طور ادامه بده
- خروجی چندکلاسه با Softmax روی ده‌ها هزار توکن – دقیقا معماری آشنای جلسه‌ی ۷!

چرا «حدس کلمه‌ی بعد» به هوش می‌رسد؟

- برای حدس خوبِ «پایتخت فرانسه ... است» باید جغرافیا بدانی
- برای ادامه‌ی «علی کلید را برداشت تا در ... را باز کند» باید داستان را بفهمی
- برای تکمیل کد برنامه‌نویسی، باید منطق برنامه را دنبال کنی
- حدس‌زدن خوب، مدل را مجبور به فشردن دانش دنیا در وزن‌هایش می‌کند
- یک هدف ساده + مقیاس عظیم = توانایی‌های شگفت‌انگیز

مقیاس: بزرگ یعنی چقدر؟

هزینه

هزاران GPU، ماه‌ها شبانه‌روز
ده‌ها میلیون دلار برای یک بار آموزش
به همین دلیل فقط چند بازیگر بزرگ داریم

داده و مدل

آموزش روی تریلیون‌ها توکن از متن اینترنت
صدها میلیارد پارامتر – همان وزن‌های آشنا
همان گرادیان کاهشی، فقط بسیار عظیم

!هر سه را می‌شناسید – Transformer، (پیش‌آموزش‌دیده) Pretrained، (مولد) Generative: یعنی GPT

سه مرحله‌ی ساخت یک دستیار

۱. پیش‌آموزش

حدس توکن بعدی
روی کل اینترنت –
دانش خام شکل می‌گیرد

۲. آموزش دستیاری

یادگیری از هزاران گفت‌وگوی
نمونه‌ی انسانی:
«این‌طور پاسخ بده»

۳. بازخورد انسانی

انسان‌ها پاسخ‌ها را رتبه می‌دهند؛
مدل به سمت پاسخ‌های بهتر
تنظیم می‌شود (RLHF)

مرحله‌ی ۳ همان جایی است که «چاپلوسی» هم می‌تواند نشت کند – جلسه‌ی اول را به یاد دارید؟

چرا هر بار جوابی متفاوت می‌گیرید؟

دما (Temperature)

دمای پایین: محافظه‌کار و قابل پیش‌بینی
دمای بالا: خلاق، متنوع – و پرت‌وپلاتر!
ابزار شما برای تنظیم خلاقیت مدل

نمونه‌گیری

مدل احتمال می‌دهد، نه جواب قطعی
خروجی از توزیع احتمال «قرعه‌کشی» می‌شود
توکن‌های محتمل‌تر، شانس بیشتری دارند

توانایی‌های نوظهور: هدیه‌ی مقیاس

- با بزرگ‌تر شدن مدل، توانایی‌هایی «ناگهان» ظاهر شدند که کسی برایشان برنامه نریخته بود
- ترجمه، خلاصه‌سازی، نوشتن کد، استدلال چندمرحله‌ای – همه از دلِ حدس توکن بعدی
- یادگیری در لحظه (In-context): چند مثال در پرامپت بدهید، الگو را می‌گیرد – بدون آموزش مجدد!
- «قدم به قدم فکر کن» بگویید تا مسائل استدلالی را بهتر حل کند
- مرز علم همین جاست: هنوز کسی دقیق نمی‌داند این توانایی‌ها چطور پدید می‌آیند

معماری RAG: پاسخ مستند

به جای اینکه مدل از حافظه‌اش حدس بزند، اول از منابع شما می‌خواند:



بزرگ‌ترین مزیت: وقتی سند عوض شد، فقط پایگاه را به‌روز کنید – نه مدل را.

جزئیات عملی RAG

نکته	گزینه‌ی رایج	تصمیم
خیلی کوچک: بی‌بافت – خیلی بزرگ: نویزی	۳۰۰ تا ۸۰۰ توکن	اندازه‌ی قطعه
تا جمله‌ای وسط دو قطعه گم نشود	۱۰ تا ۲۰ درصد	همپوشانی قطعه‌ها
بیشتر یعنی پرامپت شلوغ‌تر	۳ تا ۸	تعداد قطعه‌ی بازیابی
کیفیت بازیابی از کیفیت مدل مهم‌تر است	چندزبانه برای فارسی	مدل تعبیه
بهترین قطعه‌ها بالا بیایند	با مدل کوچک	رتبه‌بندی مجدد

بیشتر شکست‌های RAG از بازیابی بد است، نه از مدل – اول بازیابی را بسنجید.

Fine-tune یا RAG؟

Fine-tune برای «بلد بودن»

قالب خروجی خاص، لحن سازمانی
مهارتی که توضیحش سخت است
کاهش طول پرامپت و هزینه

RAG برای «دانستن»

اطلاعات، اسناد، حقایق به روز
قابل ردیابی و ارجاع دهی
به روزرسانی فوری

در عمل بیشتر سامانه‌های جدی هر دو را با هم دارند.

تطبیق ارزان با چند خط: LoRA

به جای تغییر میلیاردها وزن، ماتریس‌های کوچک کنارشان می‌گذاریم:

```
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(r=16, lora_alpha=32,
                 target_modules=['q_proj', 'v_proj'],
                 lora_dropout=0.05)

model = get_peft_model(base_model, cfg)
model.print_trainable_parameters()
# trainable: 4.2M || all: 7,000M || 0.06%

# آموزش مثل همیشه، فقط ۰,۰۶٪ پارامترها
# خروجی: یک فایل چندمگابایتی آداپتور
```

رتبه‌ی ماتریس‌های کوچک – هرچه بیشتر، ظرفیت بیشتر: $r=۱۶$

فقط لایه‌های توجه هدف قرار می‌گیرند

کمتر از یک درصد پارامترها آموزش می‌بینند

آداپتور را می‌توان جدا ذخیره و جابه‌جا کرد

چند آداپتور برای چند کار، روی یک مدل پایه

با کوانتیزه‌سازی چهاربیتی، تطبیق مدل هفت‌میلیاردی روی یک GPU معمولی ممکن می‌شود.

بخش اصلی امروز

از دستپاری که حرف می‌زند،
تا عاملی که کار انجام می‌دهد

عامل (Agent) دقیقا چیست؟

- برنامه‌ای که در یک حلقه، مدل زبانی را صدا می‌زند و ابزارهایی را که مدل انتخاب می‌کند اجرا می‌کند
- تا وقتی به هدف برسد یا به سقف تعیین‌شده بخورد
- تفاوت کلیدی با چت‌بات: عامل «اقدام» می‌کند – فایل می‌خواند، کد اجرا می‌کند، ایمیل می‌فرستد
- مدل تصمیم می‌گیرد چه کاری بعدی است؛ برنامه‌ی شما آن را اجرا می‌کند و نتیجه را برمی‌گرداند
- همین حلقه‌ی ساده، هسته‌ی همه‌ی عامل‌های امروزی است

سه سطح: چتبات، گردش کار، عامل

عامل

مدل خودش مسیر را انتخاب می‌کند
تعداد گام‌ها از پیش معلوم نیست
منعطف اما گران‌تر و لغزنده‌تر

گردش کار (Workflow)

مسیر از پیش تعیین شده
مدل در چند نقطه‌ی مشخص
قابل پیش‌بینی و ارزان

چتبات

ورودی ← پاسخ
بدون اقدام بیرونی
شما هر قدم را هدایت می‌کنید

توصیه‌ی عملی: اگر مسیر کار مشخص است، گردش کار بسازید؛ عامل را برای مسائل باز نگه دارید.

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	«گزارش فروش سه‌ماهه را بساز و برایم ایمیل کن»
۱. فکر	...
۲. انتخاب ابزار	...
۳. مشاهده	...
۴. فکر دوباره	...
۵. ابزار بعدی	...
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب‌به‌قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	«گزارش فروش سه‌ماهه را بساز و برایم ایمیل کن»
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	...
۳. مشاهده	...
۴. فکر دوباره	...
۵. ابزار بعدی	...
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	«گزارش فروش سه‌ماهه را بساز و برایم ایمیل کن»
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	...
۴. فکر دوباره	...
۵. ابزار بعدی	...
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	«گزارش فروش سه‌ماهه را بساز و برایم ایمیل کن»
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	...
۵. ابزار بعدی	...
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	«داده آمد؛ حالا باید نمودار بسازم»
۵. ابزار بعدی	...
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	«داده آمد؛ حالا باید نمودار بسازم»
۵. ابزار بعدی	اجرای کد پایتون برای رسم نمودار
۶. مشاهده	...
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	«داده آمد؛ حالا باید نمودار بسازم»
۵. ابزار بعدی	اجرای کد پایتون برای رسم نمودار
۶. مشاهده	فایل نمودار ساخته شد
۷. ابزار پایانی	...
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	«داده آمد؛ حالا باید نمودار بسازم»
۵. ابزار بعدی	اجرای کد پایتون برای رسم نمودار
۶. مشاهده	فایل نمودار ساخته شد
۷. ابزار پایانی	ارسال ایمیل با پیوست
پایان	...

حلقه‌ی عامل، قاب به قاب

عامل یعنی حلقه‌ای که تا رسیدن به هدف تکرار می‌شود: فکر ← اقدام ← مشاهده

هدف کاربر	«گزارش فروش سه‌ماهه را بساز و برایم ایمیل کن»
۱. فکر	مدل نقشه می‌ریزد: اول داده، بعد نمودار، آخر ایمیل
۲. انتخاب ابزار	فراخوانی ابزار پایگاه داده با پرس و جوی SQL
۳. مشاهده	۱۲۴ سطر داده برمی‌گردد – مدل نتیجه را می‌بیند
۴. فکر دوباره	«داده آمد؛ حالا باید نمودار بسازم»
۵. ابزار بعدی	اجرای کد پایتون برای رسم نمودار
۶. مشاهده	فایل نمودار ساخته شد
۷. ابزار پایانی	ارسال ایمیل با پیوست
پایان	هدف محقق شد – مدل حلقه را می‌بندد

ابزار چطور تعریف می‌شود؟

شما ابزار را با توضیح و امضای دقیق معرفی می‌کنید؛ مدل تصمیم می‌گیرد کی صدایش بزند:

```
tools = [{
  'name': 'search_orders',
  'description': 'جست‌وجوی سفارش‌های یک مشتری',
  'input_schema': {
    'type': 'object',
    'properties': {
      'customer_id': {'type': 'string'},
      'since': {'type': 'string',
        'description': 'تاریخ شمسی'}
    },
    'required': ['customer_id']
  },
  'required': ['customer_id']
}]
```

توضیح ابزار مهم‌ترین بخش است – مدل از روی آن تصمیم می‌گیرد

الگوی ورودی (schema) تضمین می‌کند پارامترها معتبرند

مدل به جای متن آزاد، یک فراخوانی ساختاریافته برمی‌گرداند

برنامه‌ی شما آن را اجرا و نتیجه را به مدل پس می‌دهد

ابزار بد توضیح داده شده = عاملِ سردرگم

قاعده: ابزارها را کم، واضح و بدون همپوشانی نگه دارید.

کل حلقه‌ی عامل در بیست خط

این هسته‌ی هر عامل است – بقیه‌ی فریم‌ورک‌ها روی همین سوارند:

```
messages = [{'role': 'user', 'content': goal}]

for step in range(MAX_STEPS):
    reply = model.chat(messages, tools=tools)

    if reply.tool_calls:
        for call in reply.tool_calls:
            result = TOOLS[call.name] (**call.args)
            messages.append({
                'role': 'tool',
                'tool_call_id': call.id,
                'content': str(result)})
    else:
        print(reply.text)    # پاسخ نهایی
        break
```

سقف ایمنی – عامل نباید بی‌نهایت بچرخد: MAX_STEPS

هر نتیجه‌ی ابزار به تاریخچه اضافه می‌شود

مدل در گام بعد، نتیجه را می‌بیند و تصمیم تازه می‌گیرد

نبودِ tool_call یعنی: کار تمام است

همین چند خط، تفاوت چت‌بات و عامل است

بله، واقعا همین است – سختی کار در طراحی ابزارها، حافظه و ارزیابی است، نه در این حلقه.

حافظه‌ی عامل: سه لایه

بلندمدت

پایگاه برداری یا پایگاه داده
دانش سازمانی و تجربه‌های قبلی
با جست‌وجو بازیابی می‌شود

دفترچه‌ی کار

یادداشت‌های میانی روی فایل
برنامه، پیشرفت، نتایج
عامل خودش می‌نویسد و می‌خواند

کوتاه‌مدت

تاریخچه‌ی همین گفت‌وگو
در پنجره‌ی متن
محدود و گران

اشتباه رایج: ریختن همه چیز در پنجره‌ی متن. حافظه‌ی ساختاریافته ارزان‌تر و دقیق‌تر است.

برنامه‌ریزی و تجزیه‌ی کار

- برای کارهای بزرگ، اول نقشه: عامل کار را به زیرکارهای کوچک می‌شکند
- نقشه را می‌نویسد، اجرا می‌کند و بعد از هر گام آن را به‌روز می‌کند
- زیرعامل‌ها: هر زیرکار به یک عامل تازه با زمینه‌ی تمیز سپرده می‌شود
- مثال: عامل تحقیق، برای هر منبع یک زیرعامل می‌فرستد و خلاصه‌ها را جمع می‌کند
- **مزیت مهم: پنجره‌ی متن عامل اصلی شلوغ نمی‌شود**

استاندارد اتصال عامل به ابزارها: MCP

- مشکل: هر عامل باید برای هر سرویس (ایمیل، پایگاه داده، تقویم) اتصال جدا بنویسد
- پروتکل زمینه‌ی مدل (MCP) یک زبان مشترک تعریف می‌کند: سرویس یک بار «سرور» می‌شود
- بعد هر عاملی، بدون کد اضافه، می‌تواند به آن وصل شود
- مثل USB برای ابزارهای هوش مصنوعی: یک درگاه، صدها دستگاه
- اکوسیستم امروز پر از سرورهای آماده است – از گیت‌هاب تا پایگاه داده‌های سازمانی

عواملها در عمل: سه نمونه‌ی واقعی

عامل عملیات

ایمیل و تقویم و سامانه‌ها
کارهای تکراری اداری
با تایید انسان در نقاط حساس

عامل تحقیق

ده‌ها منبع را می‌خواند
ادعاها را راستی‌آزمایی می‌کند
گزارش با ارجاع می‌سازد

عامل کدنویس

مخزن کد را می‌خواند
تغییر می‌دهد و تست اجرا می‌کند
تا وقتی تست‌ها سبز شوند

الگوی مشترک: محیط محدود، ابزارهای مشخص، و انسان در حلقه برای اقدام‌های برگشت‌ناپذیر.

